

InterviewEdge

AI-powered placement preparation for students

PROJECT REPORT + PRESENTATION BRIEF

Prepared for the 7 October 2026 presentation. Based on repository documentation and current source code checked on 6 October 2026.

PRODUCT	CURRENT BUILD	PRESENTATION FOCUS
Students practise interviews, computer-science tests and spoken English in one app.	Java/XML Android client; local Python/FastAPI, PostgreSQL, Ollama and whisper.cpp services.	Demonstrate a five-question test; explain AI scoring, progress, safeguards and remaining validation.

KEY MESSAGE

One cycle connects practice, evidence-based feedback, a shared competency profile and the next recommended activity.

What InterviewEdge does

A student-facing placement practice application with one shared record of strengths and gaps.

InterviewEdge serves students preparing for internships and entry-level software roles. The application brings together technical/HR interview practice, DSA/DBMS/OS tests and English self-introduction practice. Each activity produces saved feedback and, when eligible, updates progress and recommendations. [S1-S3]

The product loop

1. Choose	2. Practise	3. Review	4. Improve
Select a role, topic or prompt.	Answer a question, take a test or record speech.	See a score, evidence and specific feedback.	Inspect progress and take the suggested next step.

What is built today

- Local username/password accounts and recovery codes; profile, goals, consent, export and deletion.
- Android screens for Home, Practice, Progress and Profile, plus interview, test, recording and report flows.
- FastAPI endpoints, PostgreSQL migrations and jobs, starter content, local model scoring and transcription adapters.
- Scoring API with authenticated request/status endpoints, server-owned rubrics and retries.

HOW TO FRAME THE RESULT

The system is a working local prototype with tested backend journeys. It is not yet the final faculty-reviewed release; content review, wider AI calibration and full emulator accessibility testing remain.

One-minute explanation

'InterviewEdge lets a student practise three placement skills in one place. The Android app talks to a Python service on the laptop. That service saves attempts in PostgreSQL, checks objective answers against reviewed keys, and uses a local AI model for open-ended feedback. Eligible results update a common progress score and trigger focused recommendations.'

Why the project exists

Students often practise technical knowledge, interviewing and speaking in separate tools, making progress hard to connect.

The proposed benefit is a common competency profile. For example, a weak SQL explanation in an interview and a missed DBMS question can both contribute to the student's SQL practice needs. The present recommendation engine is rule-based; the product does not claim to predict placement outcomes. [S1-S3]

Audience and role presets

Audience / role	Initial focus
Student user	Practice, review feedback, track goals and choose the next activity.
Java Developer	Java language, object-oriented programming, collections, exceptions and SQL.
Backend Developer	APIs, HTTP, SQL, transactions and backend design.
Software Engineer	DSA, complexity, object-oriented programming, DBMS and OS.
Content administrator	Review, publish, retire and correct the question bank; review provisional answers.

Release boundary

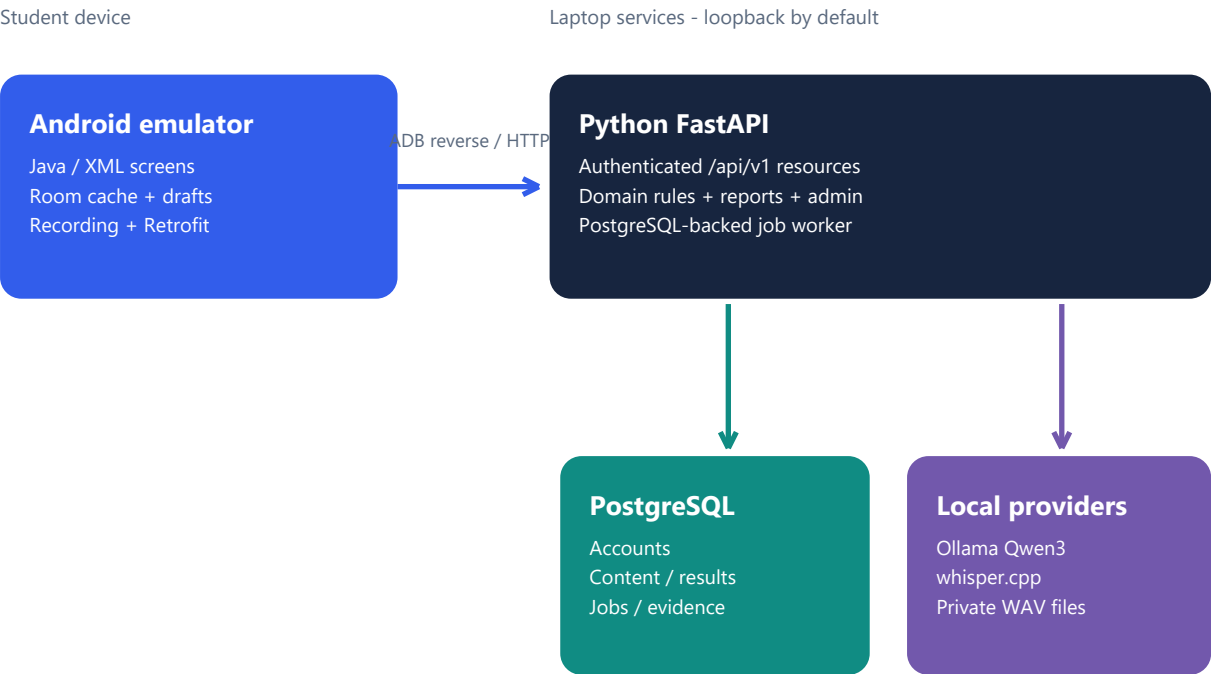
The current scope uses recorded voice one answer at a time. Resume-based interviews, uninterrupted voice chat, adaptive tests, student code execution, recruiter dashboards and institution analytics are future work. The architecture keeps role, content, media and AI providers replaceable. [S1-S3]

DECISION CHANGE DURING DEVELOPMENT

The original design mentioned Firebase and a Java/Spring backend. The implemented project uses local PostgreSQL accounts with recovery codes and a Python/FastAPI backend. The former Java backend remains archived for reference; it is not the runtime service.

How the pieces connect

A native app sends authenticated requests to one local backend, which owns business rules, persistence and AI access.



Only the backend talks to the database, AI runtime and recording store.

Responsibilities

Layer	Responsibility
Android / Java XML	Screens and navigation; Retrofit/OkHttp requests; Room cached reports and drafts; WorkManager retries; on-device PCM recording.
FastAPI / Python	Authentication, ownership checks, activity rules, verified test scoring, AI response validation, progress calculations and admin APIs.
PostgreSQL	Accounts, content versions, attempt/answer state, durable jobs, competency evidence, snapshots and audit records.
Ollama + whisper.cpp	Local Qwen3:4b text evaluation/follow-ups and local English speech transcription; media files stay in private laptop storage.

The emulator's localhost is not the laptop's localhost. **ADB reverse** connects its port 8080 to FastAPI on the laptop. Authentication and objective tests can work without AI, but open-ended evaluations require the local model; voice also requires transcription. [S2, S3]

What a student sees

The interface favors one task at a time on a small phone display.

Area	Purpose and visible flow
Login / profile	Register or sign in; save eight recovery codes; set target role, skills and weekly goal; accept practice consent.
Home	Greeting, circular Practice score (or 'Start your baseline'), weekly goal and one recommended activity.
Practice	Three large paths: mock interview, subject test and self-introduction.
Progress	Module scores, competency scores and freshness, snapshots, history and up to three reasoned recommendations.
Report / privacy	Answer-level feedback, retry or review; export account data, change password, set retention or delete account.

Three primary journeys

- Interview: choose role, skills, type, difficulty, answer mode and duration -> answer turns -> AI feedback/follow-ups -> finish -> report.
- Test: choose DSA/DBMS/OS, difficulty and length -> save answers under a server deadline -> submit -> see verified solutions and scores.
- English: read the self-introduction prompt -> record -> review the transcript -> submit -> see grammar/clarity/relevance feedback -> retry.

DASHBOARD SCORE

The ring represents performance out of 100. Weekly completion is displayed separately. A new account has no overall number until all three modules have eligible evidence. It is an educational practice indicator, not a hiring prediction.

AI mock interview

A role-specific session with technical, HR or mixed questions and answer-aware follow-ups.

The student chooses Java Developer, Backend Developer or Software Engineer; topic skills; easy, medium or hard; text or recorded voice; and a 10-, 20- or 30-minute answer budget. Session state and answer timing are stored on the server. AI processing time does not consume the answer budget; a wall-clock cap limits abandoned sessions. [S3, S5]

What happens after a text or transcribed answer

- The answer is accepted once with a submission key and stored against the current turn.
- A PostgreSQL job asks local Ollama to evaluate the answer against a server-held reference and rubric.
- For a scorable answer, the model may ask a distinct follow-up about the student's claim, with at most two follow-ups per topic.
- If follow-up generation fails, the saved evaluation is kept and the session can continue with reviewed seed content.
- The report contains question-by-question feedback, evidence excerpts, strengths, improvements and a practice score when scorable answers exist.

Interview rubric

Technical	HR
Correctness 40%; reasoning 30%; clarity 20%; relevance 10%.	Relevance 30%; structure 25%; clarity 30%; reflection 15%.

LIMITATION TO STATE

The role presets and three difficulty choices exist, but difficulty-specific interview behavior needs further validation. The included 12 interview prompts are starter content, not the final faculty-reviewed prompt bank.

Subject and DSA tests

Verified content determines objective marks; AI supports open-ended explanations and short-answer feedback.

The content catalogue covers DSA, DBMS and Operating Systems. The engine supports MCQ, code-output and short-answer items, topic selection, easy/medium/hard levels, fixed test lengths of 5/10/15, answer saving, review markers and a server-owned deadline. Active test responses omit the answer key. [S3, S5]

Scoring and review

- MCQ and code-output responses are compared with the published answer key. No AI call is required for these marks.
- A short answer is evaluated against a server-held reference and review criteria using correctness 75% and clarity 25%.
- A successful AI short-answer mark is provisional and does not update progress until an administrator reviews it.
- If AI is unavailable or the answer cannot be scored, the item remains pending with null points; objective subtotals remain visible.

Content and current demo reality

Implemented	Current limitation
45 published starter objective questions and 12 starter interview prompts are present. The database supports immutable question versions and reviewed publication.	The starter bank has five selectable questions per subject and difficulty. A 10- or 15-question test at one difficulty will report insufficient content until the bank is expanded. Use a five-question test in the demo.

WHAT TO DEMONSTRATE

Start a five-question DSA test, answer the questions, submit, open the reviewed explanation and show the updated Test module score and recommendation.

English speaking practice

The first activity is a recorded self-introduction with transcript-based feedback and retry comparison.

The prompt asks for an introduction, one project, the student's contribution and a career goal. The student records up to two minutes of audio, plays it back, reviews the transcript generated by whisper.cpp and submits the confirmed text. Individual files use mono 16 kHz 16-bit WAV; private media storage and user ownership checks protect playback. [S2, S3, S5]

Feedback available today

AI-scored text dimensions	Recorded-text metrics
Grammar 30%; clarity 35%; relevance 35%. Feedback includes verbatim evidence, strengths, improvements and suggested wording.	Word count, approximate words per minute and filler count come from the raw transcript; the report flags any edits made to the confirmed transcript.

The user can try the same prompt again and compare a later score with the earlier attempt. Retry grouping prevents immediate repeats from inflating the overall practice index. The scorer does not claim to measure pronunciation, accent, detailed pauses or voice quality from text. [S2, S3]

Voice pipeline

Record WAV -> authenticated upload -> durable transcription job
-> raw transcript -> student confirmation -> AI evaluation job
-> report + comparison -> eligible competency evidence

LIMITATIONS TO STATE

The first English activity is self-introduction. Broader accent/noise validation and full emulator microphone interruption testing are still release work. Speech quality is not inferred from transcript text.

How a score is produced

The model supplies bounded rubric dimensions and feedback; the backend validates them and calculates the number.

Ollama runs the local **qwen3:4b** model with structured JSON output. A dimension is rated 0 (incorrect/absent), 1 (major gaps), 2 (partly correct), 3 (mostly correct) or 4 (complete). The backend applies fixed, versioned weights and calculates **sum(dimension rating x weight) x 25**. [S2, S3]

Assessment	Dimensions / weights
Technical	Correctness .40; reasoning .30; clarity .20; relevance .10
HR	Relevance .30; structure .25; clarity .30; reflection .15
English	Grammar .30; clarity .35; relevance .35
Short answer	Correctness .75; clarity .25

Example: technical ratings 4, 2, 3, 4 produce $(4 \times .40 + 2 \times .30 + 3 \times .20 + 4 \times .10) \times 25 = 80/100$. The numbers shown to students are practice ratings, not validated placement probabilities.

Validation and failures

- Evidence must be copied from exact spans of the submitted answer. Extra fields, invalid types, non-finite values and ratings outside 0-4 are rejected.
- The model is instructed to grade understandable but wrong answers as low-scoring, not exclude them as unscorable.
- Unintelligible or unrelated answers can be unscorable with a null score. Missing AI is a failed/pending job, not an invented zero.
- Rubric and scoring versions, model identifier and evaluation time are stored with feedback; repeat requests reuse the accepted job/report.

From three modules to one profile

Only eligible completed work contributes to the practice summary.



Practice score example

Interview 75 x 40% = 30.0

Tests 68 x 40% = 27.2

English 74 x 20% = 14.8

Total: 72.0

The current formula is **40% Interview + 40% Tests + 20% English**. The example above gives 72.0/100. The application shows no overall score until each module has eligible evidence, and initially labels it an 'Early estimate.' [S1-S3]

Competency evidence

Each eligible activity can update one or more topic-level competencies. The backend uses the latest five eligible activity contributions per competency; immediate retries with the same group contribute once. It stores historical snapshots, last assessment dates and a flag for evidence older than 30 days. Regrading replaces previous evidence rather than adding a duplicate. [S3]

Personalized recommendations

Rules first ask the student to build any missing module baseline. They then sort weaker competencies into up to three practice suggestions, each with a reason and a destination module. A suggestion can be dismissed for three days. The Home screen surfaces one next action. These are explainable rules, not a learned recommendation model. [S3, S5]

INTERPRETATION

The practice score combines recent assessed performance. The weekly goal measures completed sessions. Evidence coverage measures how much practice supports the number. Keep these three ideas separate when explaining the dashboard.

How records relate

The schema uses activity as the shared anchor for tests, interviews and English attempts.

Entity group	Key role
edge_user, local_account, login_session, recovery_code, profile	Local identity, password/session hashes, recovery and student preferences.
role_catalog, topic, question, interview_seed	Role presets, subject/topic taxonomy, versioned questions and interview seed prompts.
activity, test_attempt, test_item	Common activity record and saved test deadline, response, score and explanation.
interview_session, interview_turn	Answer-time budget, server session state, prompts, saved answers and evaluations.
media, english_attempt	Private recording metadata, raw/confirmed transcripts and retry links.
job, progress_activity, competency_evidence, progress_snapshot	Durable processing, eligible module scores, competency contribution and trend history.

Six SQL migrations build the current schema and starter catalogue. Primary/foreign keys tie module records to an owning user and activity; unique keys help prevent duplicate submissions and progress entries. Question correction creates a new version. The Python migration runner recognizes earlier Flyway history, so it can use existing data from the former Java backend. [S3, S4]

Recordings remain on the laptop filesystem behind authenticated media endpoints; the phone keeps a local Room cache for reports and drafts. Database backups and media backups must be kept together. [S3]

The backend controls every score

The phone cannot submit a score, answer key, reference answer or rubric weight.

Endpoint family	Purpose
/api/v1/auth and /api/v1/me	Register/login/recover, profile, export and delete.
/api/v1/catalog	Roles, skills, subjects and activity options.
/api/v1/tests/attempts	Create, save, submit and review subject tests.
/api/v1/interviews	Create, answer, finish and review interview reports.
/api/v1/media and /api/v1/english/attempts	Upload/own recordings, transcribe, submit and retrieve speaking feedback.
/api/v1/activities/{id}/scoring	POST {} to request or retry scoring; GET to poll state, score and report.
/api/v1/jobs, /dashboard, /progress, /admin	Job retry, student summaries and restricted content review.

The scoring endpoint returns HTTP 202 for queued work and HTTP 200 for a reusable completed result. Its states include NOT_REQUESTED, QUEUED, RUNNING, FAILED, COMPLETED and NEEDS_REVIEW. A POST reuses the same job; the client polls GET. At most five processing attempts are allowed. [S2, S3]

Security essentials

Passwords use bcrypt; session and recovery tokens are stored as hashes. Sessions expire after seven days; recovery rotates codes and revokes old sessions. Authenticated ownership checks guard activity, reports and media IDs. Administrator access is configured server-side. The local development services bind to loopback, and Android uses a development port forward. HTTPS and deployment-specific settings are needed for ordinary network use. [S3]

What has been tested

Tests prove defined software behavior; broader educational and device validation remains open.

Evidence	Result and scope
Automated Python suite	52 passed, 7 live checks skipped in the standard run. Includes account recovery, ownership, duplicate jobs, retry limits, AI schema checks, review races, score replacement and all role/subject combinations.
Opt-in live Ollama checks	7 passed with synthetic inputs: technical, HR, English, short answer, prompt-injection rejection, correct/partial/wrong ordering, and an end-to-end scoring endpoint path.
Android build	assembleDebug and lintDebug passed before the latest scoring-only backend changes; full emulator journeys still need a final run.
Historical speech feasibility	whisper.cpp base.en transcribed an ~11-second reference clip; measured 10.23 s process wall time and ~301.6 MiB sampled peak working set.

One live scoring comparison returned **85** for a correct technical answer, **67.5** for a partial answer and **0** for an incorrect answer. A five-item endpoint check combined four correct objective answers and one 75-point AI short answer for **95**; this result remained provisional until review. Individual synthetic outcomes are not a claim of measurement validity. [S6]

RELEASE-QUALITY WORK STILL REQUIRED

Calibrate at least 60 reviewer-scored examples; test several speakers, accents and noise conditions; complete small-screen/accessibility and interruption checks; verify backup/restore and a full faculty walkthrough.

A practical demo plan

Use verified paths and have a recovery line ready if local services are slow.

Time	Show / say
0:00-1:00	Problem, student audience and three-module practice loop.
1:00-2:00	App navigation and the empty/early dashboard score; explain that 40/40/20 is a practice index.
2:00-4:00	Profile and a five-question DSA test. Submit and show explanations and progress update.
4:00-6:00	Interview setup and one text answer. Show queued evaluation, follow-up or saved report.
6:00-7:30	English self-introduction recording/transcript/report, if microphone and transcription have been prechecked.
7:30-9:00	Architecture, data ownership, scoring safeguards and measured tests.
9:00-10:00	Known gaps, faculty review plan and questions.

Start local services before opening the emulator

```
# PowerShell, from the project root
.\scripts\start-local-ai.ps1
.\scripts\start-local.ps1 -UseWorkspaceDatabase
# In a second terminal after the emulator boots
.\scripts\connect-emulator.ps1
```

Then choose the **android** project in Android Studio, select the **app** run configuration and the running emulator, and press Run. Confirm <http://127.0.0.1:8080/health> on the laptop first. The backend terminal must remain open. [S3]

DEMO SAFEGUARDS

Use a five-question test at one difficulty. Keep a completed report ready in case model loading is slow. Do not present an illustrative report as a live result. Check recording permission and local AI/transcription before the audience arrives.

Questions the panel may ask

Short responses that make the project clear without overstating the current build.

Why Python rather than Spring Boot?

The team chose FastAPI for a smaller backend codebase and faster iteration. The Android app remains native Java; PostgreSQL preserves the data. The old Java server is archived, and Python now owns its migrations and admin assets.

Why PostgreSQL if Android already has SQLite?

SQLite/Room is useful for one device's drafts and cached reports. PostgreSQL holds accounts, shared content, jobs and authoritative scores on the laptop, so a device change does not redefine the user's records.

Can students manipulate AI scores?

Requests use stored answers and server-held references. The model returns rubric dimensions, then the backend validates type, bounds and exact evidence before computing the score. Provisional short-answer marks wait for review.

Is it fully offline or free?

The local setup uses no paid services. The emulator still needs a connection to the laptop service. Once installed, local account operations do not need Firebase or public internet.

Is the score a placement probability?

No. It is a weighted practice summary. Coverage, recency and review status matter, and broader scoring calibration has not yet been completed.

What remains before faculty review release?

Expand and independently review the content bank; run reviewer-scored AI evaluation; test voice and UI on the emulator; validate backups, retention, score-version comparisons and complete account controls.

Where these facts come from

Repository files reviewed on 6 October 2026. The product blueprint records the original intent; current implementation and status files take precedence.

Reference	Repository path / interpretation
S1 - Product intent	design/InterviewEdge-Project-Blueprint.md; earlier core application plan in the project conversation. Original Firebase/Spring suggestions are historical.
S2 - Scoring contract	docs/SCORING-API.md; backend-python/app/ai.py; backend-python/app/scoring.py; backend-python/tests/test_ai.py; backend-python/tests/test_live_scoring.py.
S3 - Current implementation	README.md; docs/SETUP.md; docs/IMPLEMENTATION-STATUS.md; backend-python/app/main.py; backend-python/app/security.py; android/app/src/main/java/com/interviewedge/.
S4 - Database and content	backend-python/migrations/V1__core.sql through V6__starter_content.sql; backend-python/app/db.py.
S5 - Mobile UI and journeys	android/app/src/main/java/com/interviewedge/ui/screens/; android/app/src/main/java/com/interviewedge/ui/ScoreRing.java; android/app/build.gradle.
S6 - Verification	docs/BENCHMARKS.md; backend-python/tests/test_scoring_api.py; backend-python/tests/test_core.py; Android build/lint status in docs/IMPLEMENTATION-STATUS.md.

Source hygiene

The report distinguishes current code, verified tests, historical feasibility readings and proposed release goals. Example scores in diagrams are illustrative unless explicitly described as recorded test outcomes. UI concept images from the original blueprint are not claimed to be the present app.

Suggested final line

CLOSE

'InterviewEdge already connects practice, scoring and progress across three skill areas. Our next step is to validate its content and AI feedback with reviewers, then finish device and accessibility testing before calling it a release.'